МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ федерального государственного бюджетного образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»
Техникум Пермского института (филиала)

Практическая работа №1

по дисциплине «Поддержка и тестирование программных модулей»

Тема: «разработка модуля импорта на платформе ASP.NET Core MVC»

Студента группы ИПо-42
Светлышева Елизавета Денисовна

2026 г.

Оглавление

ВВЕДЕНИЕ 3
ГЛАВА 1. ОПИСАНИЕ ЗАДАНИЯ, ТЕХНОЛОГИЧЕСКИЙ СТЕК И АРХИТЕКТУРА ПРОЕКТА 5
1.1 Описание задания 5
1.2 Используемые технологии и инструменты 5
1.3 Архитектура приложения 6
ГЛАВА 2. ФОРМАТ CSV-ФАЙЛА И ПРАВИЛА ВАЛИДАЦИИ 8
2.1 Формат файла 8
2.2 Правила валидации полей 9
2.3 Ошибки уровня файла и строки 10
2.4 Кодировка файла 11
ГЛАВА 3. МОДЕЛИ ДАННЫХ 11
3.1 Обзор классов 11
3.2 Ключевые классы 12
ГЛАВА 4. СЕРВИСНЫЙ СЛОЙ 12
4.1 Интерфейс IProductRepository и InMemoryProductRepository 12
4.2 Класс CsvLineParser 13
4.3 Сервис ProductImportService 13
4.4 ProductFilter и MoneyFormat 15
ГЛАВА 5. КОНТРОЛЛЕР И ПРЕДСТАВЛЕНИЯ ASP.NET MVC 16
5.1 ProductsController 16
5.2 Представления 16
ГЛАВА 6. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ 17
6.1 Структура тестового проекта 17
6.2 Тесты репозитория и парсера 18
6.3 Тесты ProductImportService 19
6.4 Прочие тесты 20
ГЛАВА 7. РАБОЧИЙ ПРОЦЕСС ПРИЛОЖЕНИЯ 21
ЗАКЛЮЧЕНИЕ 25

ВВЕДЕНИЕ

В рамках практики передо мной стояла задача разработать модуль импорта на платформе ASP.NET Core MVC. Модуль читает CSV-файл с данными о товарах, преобразует строки файла в объекты C#, проверяет корректность значений, добавляет корректные товары в коллекцию в памяти приложения и показывает пользователю результат импорта: общее количество строк, количество успешно загруженных товаров, количество ошибок и список ошибок с номерами строк. Загруженные товары отображаются на отдельной странице списка.

Работа выполнена в рамках производственной практики ПП.02.01 «Осуществление интеграции программных модулей» по специальности 09.02.07 «Информационные системы и программирование» и соответствует профессиональным компетенциям ПК 2.1–2.5: разработка требований к программному модулю, его интеграция в приложение, отладка, разработка тестовых наборов и сценариев, инспектирование кода на соответствие стандартам. Индивидуальное задание выполнено по варианту 1 «Электроника», который расширяет базовую модель товара полями brand (бренд, обязательное поле), warranty_months (гарантия в месяцах, целое число от 0 до 60) и model_number (модель, может быть пустым).

Целью работы является разработка работоспособного веб-модуля, демонстрирующего слоистую организацию кода, разделение ответственности между контроллером, сервисом и хранилищем, обработку некорректных входных данных без аварийного завершения приложения и покрытие логики модульными тестами.

Для достижения цели были решены следующие задачи:

– анализ требований и описание формата CSV-файла;

– проектирование модели товара и хранилища данных в памяти;

– реализация собственного парсера строк CSV с поддержкой кавычек и пустых полей;

– реализация сервиса импорта с валидацией полей и обработкой дубликатов артикулов;

– создание MVC-интерфейса: страницы импорта и страницы списка товаров;

– разработка модульных тестов и сценариев ручного тестирования;

– самопроверка кода на соответствие базовым стандартам.

В ходе разработки реализованы четыре взаимосвязанных компонента: хранилище товаров в памяти, парсер CSV, сервис импорта и MVC-интерфейс. Дополнительно реализованы поиск по названию, артикулу, бренду и модели, фильтрация по категории, сортировка, подсчёт общего количества и стоимости товаров, очистка списка и чтение файлов в кодировке Windows-1251. Для контроля качества разработан комплект из 47 тестовых методов (54 тестовых случая) на фреймворке xUnit.

ГЛАВА 1. ОПИСАНИЕ ЗАДАНИЯ, ТЕХНОЛОГИЧЕСКИЙ СТЕК И АРХИТЕКТУРА ПРОЕКТА

1.1 Описание задания

Задание состоит в разработке модуля, который наполняет электронный магазин товарами из CSV-файла. Модуль должен читать файл построчно, проверять обязательные поля и допустимость значений, формировать список ошибок с номерами строк, добавлять корректные товары в память приложения, отображать результат импорта и список загруженных товаров. База данных в задании не используется: данные хранятся только в памяти приложения, и после его перезапуска они теряются, что допустимо по условиям задания.

При выполнении задания приняты следующие проектные решения:

– товары хранятся в репозитории, зарегистрированном как singleton, чтобы данные сохранялись между HTTP-запросами;

– дубликат артикула (SKU) пропускается, а в результат импорта добавляется ошибка «Артикул товара уже существует»;

– CSV читается без внешних библиотек собственным парсером с поддержкой кавычек и запятых внутри полей;

– путь к файлу указывается в форме: абсолютный или относительно папки wwwroot;

– повторный запуск импорта допускается в любой момент и не приводит к падению приложения.

1.2 Используемые технологии и инструменты

Для реализации выбран стек на базе .NET 8. Полный перечень технологий приведён в таблице 1.

Таблица 1 – перечень технологий

Компонент	**Технология / Инструмент**
Платформа разработки	.NET 8, язык C# (включены Nullable и ImplicitUsings)
Веб-фреймворк	ASP.NET Core MVC, представления Razor
Архитектурный паттерн	MVC (Model – View – Controller) с выделенным сервисным слоем
Хранение данных	Коллекция List<Product> в InMemoryProductRepository (singleton)
Разбор CSV	Собственный класс CsvLineParser, внешние библиотеки не используются
Модульное тестирование	xUnit 2.5.3, Microsoft.NET.Test.Sdk 17.8.0
Оформление интерфейса	Собственный файл стилей wwwroot/css/site.css
Система контроля версий	Git

Структура проекта, смотреть Рисунок 1.2.1 – Структура проекта

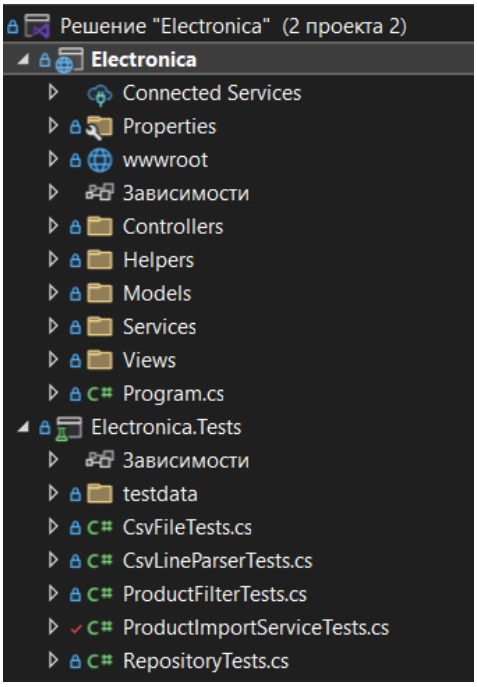


Рисунок 1.2.1 – Структура проекта

1.3 Архитектура приложения

При проектировании я разделила код на слои, чтобы контроллер не разрастался и не превращался в монолит. Бизнес-логику я вынесла в сервис, а работу с данными — в репозиторий. В итоге контроллер у меня получился "тонким": он не содержит бизнес-логики импорта, а только принимает

запрос, вызывает сервис и передаёт готовый результат в представление. Вся тяжёлая работа — чтение файла, разбор строк, валидация и создание объектов — находится в сервисе импорта. Хранение товаров я тоже отделила от представления и скрыла за интерфейсом репозитория, чтобы в будущем можно было легко заменить хранилище в памяти на базу данных. Поток данных в моём приложении выглядит следующим образом:

CSV-файл → ProductImportService (CsvLineParser, валидация, создание Product) → InMemoryProductRepository → ProductsController → Razor-представления.

Структура проекта и назначение основных файлов приведены в таблице 2.

Таблица 2 – структура проекта

Файл / папка	**Назначение**
Controllers/ProductsController.cs	Тонкий контроллер: действия Index, Import (GET и POST), Clear, Error
Models/Product.cs	Товар с полями базовой модели и варианта 1 «Электроника»
Models/ImportResult.cs, ImportError.cs	Результат импорта и описание одной ошибки
Models/ImportViewModel.cs, ProductsIndexViewModel.cs	Модели представлений страниц импорта и списка
Services/IProductRepository.cs, InMemoryProductRepository.cs	Интерфейс и потокобезопасная реализация хранилища в памяти
Services/CsvLineParser.cs	Разбор одной строки CSV с поддержкой кавычек
Services/IProductImportService.cs, ProductImportService.cs	Интерфейс и реализация сервиса импорта
Services/ProductFilter.cs	Поиск, фильтрация по категории и сортировка списка
Helpers/MoneyFormat.cs	Форматирование денежных сумм для вывода
Views/Shared/_Layout.cshtml	Общий макет: шапка с навигацией и основная область страницы
Views/Products/Index.cshtml, Import.cshtml, Error.cshtml	Страницы списка товаров, импорта и ошибки

Файл / папка	Назначение
wwwroot/csv/sample_products.csv	Пример корректного CSV-файла (11 товаров)
Program.cs	Точка входа: настройка сервисов, локализации и маршрутов
tests/Electronica.Tests	Проект модульных тестов xUnit и папка testdata с CSV-файлами

Точкой входа является файл Program.cs. В нём подключаются MVC (AddControllersWithViews) и веб-кодировщик с диапазоном UnicodeRanges.All, благодаря которому кириллица в HTML выводится как есть, а не числовыми кодами. Затем регистрируются зависимости: IProductRepository → InMemoryProductRepository и IProductImportService → ProductImportService, обе как singleton. Это принципиально важно: каждый HTTP-запрос обслуживается новым экземпляром контроллера, и при регистрации с временем жизни Scoped список товаров терялся бы при переходе со страницы импорта на страницу списка.

Вне режима разработки включён обработчик исключений, перенаправляющий пользователя на страницу /Products/Error. Через UseRequestLocalization задана культура ru-RU, чтобы числа выводились единообразно (например, 45 990,50) независимо от региональных настроек сервера. Маршрут по умолчанию — Products/Index.

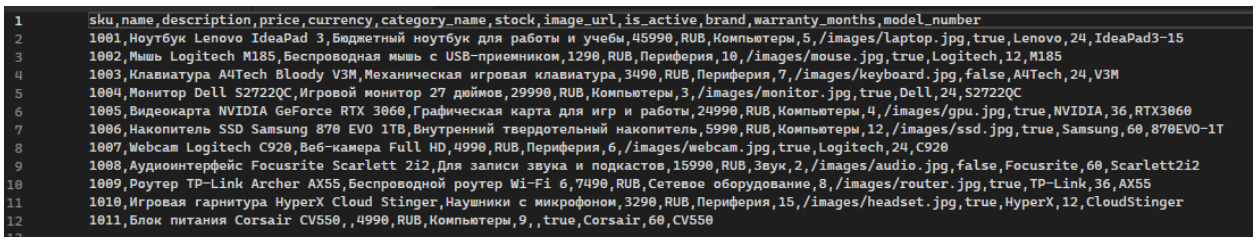
ГЛАВА 2. ФОРМАТ CSV-ФАЙЛА И ПРАВИЛА ВАЛИДАЦИИ

2.1 Формат файла

Файл представляет собой текст, в котором поля разделены запятой, а первая непустая строка содержит заголовки смотреть Рисунок 2.1.1 – Пример CSV-файла. Обязательны все 12 колонок, перечисленных в таблице 3. Заголовки ищутся без учёта регистра и пробелов; если колонка встречается несколько раз, используется первое вхождение; дополнительные колонки игнорируются. Поля, содержащие запятую, заключаются в кавычки, а кавычка внутри поля удваивается.

Таблица 3 – поля CSV-файла

Поле	**Тип**	**Обязательность**	**Описание**
sku	string	Обязательно	Артикул товара, уникальный
name	string	Обязательно	Название товара
description	string	Нет	Описание товара
price	decimal	Обязательно, больше 0	Цена товара
currency	string	Нет	Валюта; если пусто — RUB; приводится к верхнему регистру
category_name	string	Нет	Категория; если пусто — «Без категории»
stock	int	Обязательно, не меньше 0	Остаток на складе
image_url	string	Нет	Ссылка на изображение
is_active	bool	Обязательно	Активность товара: true, false, 1 или 0
brand	string	Обязательно	Бренд (вариант 1 «Электроника»)
warranty_months	int	Обязательно, от 0 до 60	Гарантия в месяцах (вариант 1)
model_number	string	Нет	Номер модели (вариант 1)



```
sku,name,description,price,currency,category_name,stock,image_url,is_active,brand,warranty_months,model_number
1001,Ноутбук Lenovo IdeaPad 3,Бюджетный ноутбук для работы и учебы,45990,RUB,Компьютеры,5,/images/laptop.jpg,true,Lenovo,24,IdeaPad3-15
1002,Мышь Logitech M185,Беспроводная мышь с USB-приемником,1290,RUB,Периферия,10,/images/mouse.jpg,true,Logitech,12,M185
1003,Клавиатура A4Tech Bloody V3M,Механическая игровая клавиатура,3490,RUB,Периферия,7,/images/keyboard.jpg,false,A4Tech,24,V3M
1004,Монитор Dell S2722QC,Игровой монитор 27 дюймов,29990,RUB,Компьютеры,3,/images/monitor.jpg,true,Dell,24,S2722QC
1005,Видеокарта NVIDIA GeForce RTX 3060,Графическая карта для игр и работы,24990,RUB,Компьютеры,4,/images/gpu.jpg,true,NVIDIA,36,RTX3060
1006,Накопитель SSD Samsung 870 EVO 1TB,Внутренний твердотельный накопитель,5990,RUB,Компьютеры,12,/images/ssd.jpg,true,Samsung,60,870EVO-1T
1007,Webcam Logitech C920,Веб-камера Full HD,4990,RUB,Периферия,6,/images/webcam.jpg,true,Logitech,24,C920
1008,Аудиоинтерфейс Focusrite Scarlett 2i2,Для записи звука и подкастов,15990,RUB,Звук,2,/images/audio.jpg,false,Focusrite,60,Scarlett2i2
1009,Роутер TP-Link Archer AX55,Беспроводной роутер Wi-Fi 6,7490,RUB,Сетевое оборудование,8,/images/router.jpg,true,TP-Link,36,AX55
1010,Игровая гарнитура HyperX Cloud Stinger,Наушники с микрофоном,3290,RUB,Периферия,15,/images/headset.jpg,true,HyperX,12,CloudStinger
1011,Блок питания Corsair CV550,,4990,RUB,Компьютеры,9,,true,Corsair,60,CV550
```

Рисунок 2.1.1 – Пример CSV-файла

2.2 Правила валидации полей

Каждая строка данных проверяется по правилам таблицы 4. Поля проверяются в фиксированном порядке: sku, name, price, stock, is_active, brand, warranty_months. Проверка не прекращается на первой ошибке, поэтому одна строка может дать несколько ошибок сразу.

Таблица 4 – правила валидации и сообщения об ошибках

Поле	**Правило**	**Сообщение об ошибке**
sku	Не пустое после удаления пробелов	Поле sku не может быть пустым
name	Не пустое после удаления пробелов	Поле name не может быть пустым
price	Число больше 0	Поле price должно быть числом больше 0
stock	Целое число не меньше 0	Остаток на складе должен быть целым числом не меньше 0
is_active	true, false, 1 или 0 без учёта регистра	Поле is_active должно иметь значение true, false, 1 или 0
brand	Не пустое после удаления пробелов	Поле brand обязательно для варианта «Электроника»
warranty_months	Целое число от 0 до 60 включительно	Гарантия должна быть целым числом от 0 до 60 месяцев

2.3 Ошибки уровня файла и строки

Помимо проверки полей, сервис формирует сообщения о проблемах с файлом целиком и со структурой строки (таблица 5). Для ошибок уровня файла используется номер строки 0, а на странице результата вместо нуля выводится слово «файл». Номера остальных ошибок физические: первая строка файла имеет номер 1, поэтому первая строка данных после заголовка имеет номер 2.

Таблица 5 – ситуации, не связанные с проверкой отдельных полей

Ситуация	**Сообщение**	**Номер строки**
Путь пустой или файл не найден	Файл не найден	0
Файл пустой или нет строк данных	Данные для импорта отсутствуют	0
Не хватает обязательных колонок	Неверные заголовки CSV	Строка заголовка
Число полей не равно числу полей заголовка	Неверное количество полей в строке	Строка данных

Ситуация	Сообщение	Номер строки
Артикул уже есть в репозитории или в этом же файле	Артикул товара уже существует	Строка данных
Ошибка ввода-вывода или нет доступа к файлу	Не удалось прочитать файл: <текст исключения>	0

2.4 Кодировка файла

Файл сначала декодируется как UTF-8 в строгом режиме (с BOM или без него). Если в файле встречаются байты, недопустимые для UTF-8, он перечитывается в кодировке Windows-1251. Благодаря этому русские названия товаров отображаются корректно независимо от того, в какой кодировке сохранён файл. Символ BOM в начале текста удаляется, а строки разделяются по любому из вариантов перевода строки: \r\n, \n или \r.

ГЛАВА 3. МОДЕЛИ ДАННЫХ

3.1 Обзор классов

Классы моделей расположены в пространстве имён Electronica.Models. Их назначение приведено в таблице 6.

Таблица 6 – классы моделей

Класс	Назначение
Product	Товар интернет-магазина (базовые поля и поля варианта 1 «Электроника»)
ImportResult	Результат импорта: число строк, число импортированных товаров, список ошибок
ImportError	Описание одной ошибки: номер строки и текст сообщения
ImportViewModel	Модель страницы импорта: введённый путь и результат импорта
ProductsIndexViewModel	Модель страницы списка: товары, категории, параметры поиска и итоги

3.2 Ключевые классы

– Product. Центральная сущность. Содержит поля Sku, Name, Description, Price (decimal), Currency, CategoryName, StockQuantity (int), ImageUrl, IsActive и поля варианта 1: Brand, WarrantyMonths и ModelNumber. Для валюты по умолчанию задано значение «RUB», для категории — «Без категории», строковые поля инициализируются пустой строкой, что исключает ссылки null при включённой проверке Nullable.

– ImportResult. Хранит TotalRows (количество непустых строк данных после заголовка), ImportedCount (количество товаров, добавленных в репозиторий) и список Errors. Свойство ErrorCount вычисляется как количество элементов списка ошибок, поэтому не может разойтись с самим списком.

– ImportError. Содержит RowNumber и Message. Номер 0 означает ошибку уровня файла.

– ImportViewModel. Передаёт в представление строку FilePath (чтобы форма сохраняла введённое значение) и необязательный объект Result.

– ProductsIndexViewModel. Содержит список товаров после применения поиска, фильтра и сортировки (Products), общее число товаров в памяти (AllProductsCount), список категорий для выпадающего списка, текущие параметры Query, Category и Sort. Вычисляемые свойства: TotalCount — число показанных товаров, TotalValue — сумма цен, StockValue — стоимость остатков (сумма произведений цены на остаток), IsFiltered — признак того, что применён поиск или фильтр.

ГЛАВА 4. СЕРВИСНЫЙ СЛОЙ

4.1 Интерфейс IProductRepository и InMemoryProductRepository

Хранилище скрыто за интерфейсом IProductRepository, который содержит методы Add(Product), GetAll(), GetBySku(string), свойство Count и

метод Clear(). Реализация InMemoryProductRepository хранит товары в List<Product> и защищает все обращения к списку блокировкой, поскольку репозиторий зарегистрирован как singleton и может использоваться несколькими запросами одновременно.

Метод Add() выбрасывает ArgumentNullException при передаче null. Метод GetAll() возвращает не сам внутренний список, а его копию (снимок), поэтому внешние изменения не влияют на хранилище. Метод GetBySku() ищет артикул без учёта регистра и крайних пробелов и возвращает null для пустого значения или если товар не найден. Метод Clear() используется кнопкой «Очистить список» и тестами.

4.2 Класс CsvLineParser

Статический класс CsvLineParser содержит единственный метод Parse(string line), который разбирает одну строку CSV на поля. Разбор выполняется посимвольно с признаком «внутри кавычек». Вне кавычек запятая завершает текущее поле; кавычка открывает режим «внутри кавычек». Внутри кавычек запятая считается обычным символом, а пара подряд идущих кавычек превращается в одну кавычку в значении поля. После разбора каждое поле обрезается по краям (trim). Пустые поля сохраняются как пустые строки, поэтому число полей строки всегда равно числу запятых вне кавычек плюс один. Разделитель и символ кавычки вынесены в константы Delimiter и Quote.

4.3 Сервис ProductImportService

Сервис реализует интерфейс IProductImportService с двумя методами: ImportFromCsv(string filePath) и ImportFromLines(IEnumerable<string> lines). Первый работает с файловой системой, второй содержит всю логику импорта и не зависит от файлов, что позволяет тестировать его без создания файлов на диске. Репозиторий передаётся через конструктор; при передаче null выбрасывается ArgumentNullException.

Метод ImportFromCsv() возвращает ошибку «Файл не найден», если путь пуст или файл отсутствует. Чтение файла обёрнуто в try/catch для IOException и UnauthorizedAccessException, поэтому проблемы с доступом превращаются в сообщение об ошибке, а не в исключение. Затем строки передаются в ImportFromLines().

Алгоритм метода ImportFromLines() состоит из следующих шагов:

1. Список строк преобразуется в List; если все строки пусты, возвращается ошибка «Данные для импорта отсутствуют».

2. Первая непустая строка считается заголовком; из неё удаляется символ BOM, поля разбираются парсером, строится словарь «имя колонки → индекс» без учёта регистра.

3. Проверяется наличие 12 обязательных колонок; при отсутствии любой из них добавляется ошибка «Неверные заголовки CSV» с номером строки заголовка, и импорт прекращается.

4. Ожидаемое число полей принимается равным числу полей заголовка; создаётся множество HashSet артикулов, импортированных в текущем файле.

5. Для каждой следующей строки: пустая строка пропускается и не увеличивает TotalRows; иначе TotalRows увеличивается на единицу, определяется физический номер строки, и строка разбирается парсером. Если число полей не совпало с ожидаемым, добавляется ошибка «Неверное количество полей в строке».

6. Закрытый метод ParseRow() проверяет все поля в установленном порядке, собирает все ошибки строки и при их наличии возвращает null; такая строка в репозиторий не попадает.

7. Для корректной строки проверяется дубликат: артикул уже есть в репозитории или во множестве импортированных артикулов текущего файла. Дубликат не добавляется, а в результат записывается ошибка «Артикул товара уже существует».

8. Если дубликата нет, товар добавляется в репозиторий, артикул — во множество, а счётчик ImportedCount увеличивается.

9. Если после обработки TotalRows равен нулю и ошибок нет (в файле был только заголовок), добавляется ошибка «Данные для импорта отсутствуют».

Разбор значений выполняется вспомогательными методами. TryParseDecimal() заменяет запятую на точку и разбирает число в инвариантной культуре, допуская знак и десятичную точку, но не разделители тысяч. TryParseInt() принимает целое число со знаком. TryParseBool() принимает значения true, false, 1 и 0 без учёта регистра. NormalizeCurrency() возвращает «RUB» для пустого значения, иначе значение в верхнем регистре; NormalizeCategory() возвращает «Без категории» для пустого значения.

Строки сообщений, значения по умолчанию и границы гарантии вынесены в константы (DefaultCurrency, DefaultCategory, MinWarrantyMonths, MaxWarrantyMonths и константы Msg…), что исключает «магические числа» и дублирование текстов в коде.

4.4 ProductFilter и MoneyFormat

Статический класс ProductFilter реализует необязательные функции задания. Метод Apply() принимает список товаров, строку поиска, категорию и режим сортировки. Поиск выполняется без учёта регистра по названию, артикулу, бренду и номеру модели. Фильтр по категории сравнивает названия без учёта регистра. Поддерживаются сортировки по названию, по цене (по возрастанию и по убыванию) и по остатку (по убыванию); при остальных значениях сохраняется порядок импорта.

Статический класс MoneyFormat форматирует суммы для вывода в культуре ru-RU: целые суммы выводятся без копеек (45 990), дробные — с двумя знаками (1 290,50), а метод WithCurrency() добавляет символ «₽» для рублей через неразрывный пробел.

ГЛАВА 5. КОНТРОЛЛЕР И ПРЕДСТАВЛЕНИЯ ASP.NET MVC

5.1 ProductsController

Контроллер получает репозиторий, сервис импорта и IWebHostEnvironment через конструктор, при передаче null любой из зависимостей выбрасывается ArgumentNullException. Бизнес-логика в контроллере отсутствует. Действия контроллера приведены в таблице 7.

Таблица 7 – действия ProductsController

Действие	**Метод HTTP**	**Назначение**
Index(q, category, sort)	GET	Показывает список товаров; применяет поиск, фильтр и сортировку через ProductFilter, формирует список категорий и итоги
Import()	GET	Показывает форму импорта с пустым результатом
Import(filePath)	POST	Определяет полный путь к файлу, вызывает ImportFromCsv() и возвращает страницу с результатом; защищено ValidateAntiForgeryToken
Clear()	POST	Очищает репозиторий и перенаправляет на список товаров; защищено ValidateAntiForgeryToken
Error()	GET	Показывает страницу непредвиденной ошибки

Метод ResolveFilePath() определяет, откуда читать файл. Пустой ввод означает отсутствие пути, и пользователю возвращается сообщение «Укажите путь к CSV-файлу». Путь вида /csv/sample_products.csv ищется в папке wwwroot; полный путь (например, C:\data\products.csv) используется как есть. Поскольку строка /csv/file.csv формально считается абсолютным путём, сначала проверяется существование файла в wwwroot, и только потом используется введённый путь.

5.2 Представления

Общий макет _Layout.cshtml содержит шапку с названием магазина и двумя пунктами навигации («Товары» и «Импорт CSV»), подсвечивающими текущую страницу, и основную область, в которую выводится содержимое

страниц. Стили подключены из файла wwwroot/css/site.css без внешних библиотек.

– Import.cshtml. Содержит форму с текстовым полем пути к CSV-файлу и кнопкой «Импортировать товары». Форма строится тег-хелперами, поэтому в неё автоматически добавляется защитный токен. Под полем расположена кнопка-подсказка, подставляющая путь /csv/sample_products.csv. После импорта показывается цветное сообщение (успех, частичный импорт или ни один товар не импортирован), таблица показателей «Всего строк», «Успешно импортировано», «Ошибки» и таблица ошибок с номерами строк. Для ошибок уровня файла вместо нуля выводится слово «файл». Ниже расположена ссылка на список товаров.

– Index.cshtml. Показывает три показателя (всего товаров, общая стоимость, стоимость остатков), панель поиска, фильтра по категории и сортировки, а также таблицу из 11 столбцов, включая бренд, гарантию и модель. Если товаров нет, выводится сообщение «Товары еще не импортированы» со ссылкой на импорт; если поиск ничего не нашёл — предупреждение. Нулевой остаток выделяется стилем, а активность отображается как «Да» или «Нет». Кнопка «Очистить список» запрашивает подтверждение.

– Error.cshtml. Показывает дружелюбное сообщение о непредвиденной ошибке и ссылку на список товаров; данные в памяти при этом не теряются.

ГЛАВА 6. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

6.1 Структура тестового проекта

Тестовый проект Electronica.Tests использует фреймворк xUnit и ссылается на основной проект. Тест-классы и их состав приведены в таблице 8. Для изоляции каждый тест создаёт собственный экземпляр InMemoryProductRepository, поэтому тесты не влияют друг на друга. Файлы из папки testdata копируются в выходной каталог сборки, и тесты обращаются к ним через AppContext.BaseDirectory.

Таблица 8 – тест-классы

Тест-класс	**Число тестов**	**Что проверяет**
RepositoryTests	8 методов (9 случаев)	Добавление, поиск по SKU (регистр, пробелы), пустой репозиторий, снимок списка, Add(null), Clear
CsvLineParserTests	5	Простая строка, пустые поля, запятая в кавычках, экранированные кавычки, trim
ProductImportServiceTests	23	Все сценарии импорта и валидации (таблица 9)
CsvFileTests	4 метода (10 случаев)	Готовые CSV из testdata, номер строки ошибки, кодировки Windows-1251 и UTF-8 с BOM
ProductFilterTests	7	Поиск, фильтр по категории, сортировки, форматирование денежных сумм

Результат выполнения модульных тестов смотреть на Рисунок 6.1.1 – Результат выполнения модульных тестов

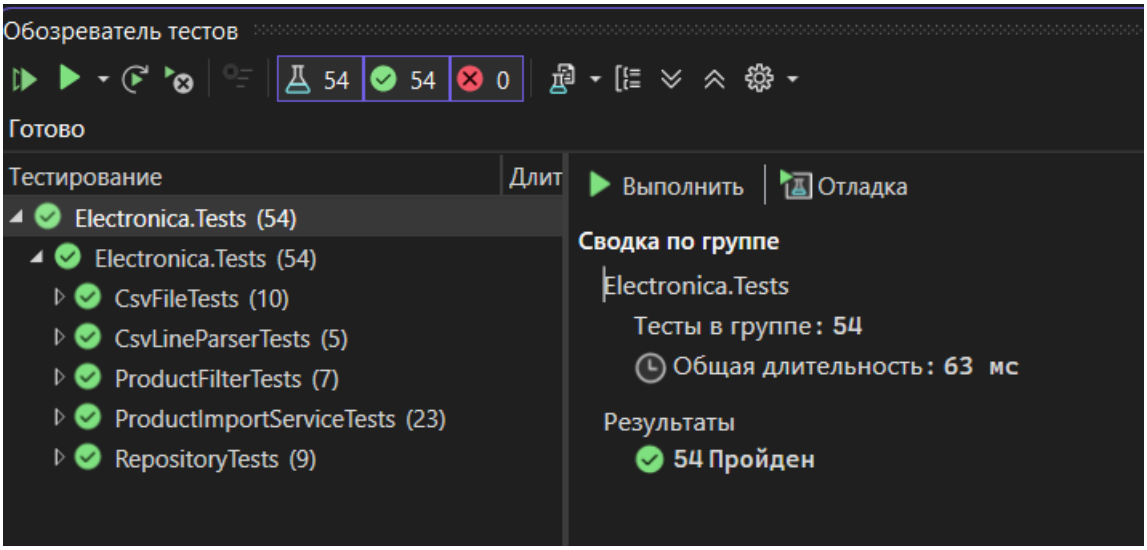


Рисунок 6.1.1 – Результат выполнения модульных тестов

6.2 Тесты репозитория и парсера

Тесты репозитория проверяют, что добавленный товар появляется в GetAll() и учитывается в Count; что GetBySku() находит товар по точному артикулу, по артикулу с пробелами по краям и без учёта регистра; что пустой репозиторий возвращает Count, равный нулю, пустой список и null при поиске; что изменение полученного списка не влияет на внутреннюю

коллекцию; что Add(null) выбрасывает исключение; что Clear() удаляет все товары.

Тесты парсера проверяют разбор строки из 12 полей без кавычек, наличие пустых третьего и восьмого полей в строке с пропущенными значениями, сохранение запятой внутри поля в кавычках («Ноутбук, игровой»), превращение удвоенных кавычек в одну (Название "Special") и удаление пробелов по краям полей.

6.3 Тесты ProductImportService

Для большинства тестов используется метод ImportFromLines(), что позволяет не создавать файлы на диске; тесты, связанные с файлами, вызывают ImportFromCsv() с временными файлами. Перечень тестов приведён в таблице 9. В колонке «Результат» через дробь указаны TotalRows / ImportedCount.

Таблица 9 – тесты сервиса импорта

№	Сценарий	Входные данные	Ожидаемый результат
1	Одна корректная строка	Заголовок и 1 товар	1/1, ошибок нет, поля товара верны
2	Несколько строк	Заголовок и 2 товара	2/2, в репозитории 2 товара
3	Файл не найден	Несуществующий путь	0/0, «Файл не найден»
4	Пустой файл	Файл без содержимого	0/0, «Данные для импорта отсутствуют»
5	Только заголовок	Одна строка заголовка	0/0, «Данные для импорта отсутствуют»
6	Неверный заголовок	Заголовок из 9 колонок	0/0, «Неверные заголовки CSV»
7	Неверное число полей	Строка из 2 полей	1/0, «Неверное количество полей в строке»
8	Пустой SKU	sku пустое	0 импортировано, «Поле sku не может быть пустым»

№	Сценарий	Входные данные	Ожидаемый результат
9	Пустое название	name пустое	0 импортировано, «Поле name не может быть пустым»
10	Отрицательная цена	price = -500	0 импортировано, ошибка по полю price
11	Нечисловая цена	price = abc	0 импортировано, ошибка по полю price
12	Отрицательный остаток	stock = -5	0 импортировано, ошибка по полю stock
13	Неверный is_active	is_active = yes	0 импортировано, ошибка по полю is_active
14	Нет бренда	brand пустое	0 импортировано, ошибка по полю brand
15	Гарантия меньше нуля	warranty_months = -1	0 импортировано, ошибка гарантии
16	Гарантия больше 60	warranty_months = 61	0 импортировано, ошибка гарантии
17	Дубликат внутри файла	Две строки с SKU 1001	2/1, «Артикул товара уже существует», остаётся первый товар
18	Дубликат с репозиторием	SKU уже есть в репозитории	Товар не добавлен, Count = 1, ошибка дубликата
19	Несколько ошибок в строке	Пустые sku, name, brand; price < 0; stock < 0; is_active = yes; гарантия 61	0 импортировано, не менее 4 ошибок, репозиторий пуст
20	Необязательные поля	Пустые description и image_url	Товар импортирован, ошибок нет
21	Пустая валюта	currency пустое	Currency = «RUB»
22	Пустая категория	category_name пустое	CategoryName = «Без категории»
23	Кириллица	Русские название, описание и категория	Значения сохранены без искажений

6.4 Прочие тесты

Класс CsvFileTests содержит параметризованный тест, который прогоняет сервис по семи готовым файлам из папки testdata (valid_electronics.csv, invalid_price.csv, duplicate_sku.csv, missing_brand.csv,

warranty_out_of_range.csv, bad_active.csv, empty_file.csv) и сверяет число строк, число импортированных товаров, число ошибок и текст первой ошибки. Отдельные тесты проверяют, что ошибка неверной цены относится к строке 2, что файл в кодировке Windows-1251 читается без искажения кириллицы и что файл в UTF-8 с BOM импортируется без ошибок.

Класс ProductFilterTests проверяет работу списка без фильтров (исходный порядок), поиск без учёта регистра, фильтр по категории, сортировку по цене в обоих направлениях, пустой результат при отсутствии совпадений и форматирование сумм (копейки выводятся только при необходимости).

ГЛАВА 7. РАБОЧИЙ ПРОЦЕСС ПРИЛОЖЕНИЯ

Работа с приложением строится вокруг двух страниц — импорта и списка товаров.

Этап 1: Запуск приложения. Приложение запускается. Открывается страница «Товары» (маршрут по умолчанию). Поскольку данные хранятся в памяти и после запуска ещё не загружены, выводится сообщение «Товары еще не импортированы» и ссылка «Перейти к импорту».

Этап 2: Импорт файла. На странице «Импорт CSV» пользователь вводит путь к файлу (например, /csv/sample_products.csv) или нажимает кнопку-подсказку и запускает импорт. Контроллер определяет полный путь и вызывает сервис. Результат отображается на той же странице: для файла sample_products.csv всего строк — 11, успешно импортировано — 11, ошибок — 0.

Этап 3: Просмотр товаров. По ссылке «Перейти к списку товаров» пользователь открывает таблицу товаров. Данные сохранились между запросами благодаря регистрации репозитория как singleton. На странице видны общее количество товаров, общая стоимость и стоимость остатков.

Этап 4: Поиск, фильтрация и сортировка. Панель над таблицей позволяет искать товары по названию, артикулу, бренду или модели,

выбирать категорию и порядок сортировки. Параметры передаются методом GET, поэтому адрес страницы с фильтром можно сохранить в закладках.

Этап 5: Повторный импорт и ошибки. При повторном импорте того же файла все артикулы уже существуют, поэтому ни один товар не добавляется, а для каждой строки выводится ошибка «Артикул товара уже существует» с номером строки. Файлы с неверной ценой, пустым брендом, гарантией вне диапазона, неверным значением is_active, а также пустой или несуществующий файл дают понятные сообщения, и приложение продолжает работать.

Этап 6: Очистка списка. Кнопка «Очистить список» после подтверждения удаляет все товары из памяти, что позволяет запустить импорт заново с чистого состояния.

Результаты ручного тестирования сценариев приведены в таблице 10.

Таблица 10 – ручные тестовые сценарии

№	**Сценарий**	**Входные данные**	**Ожидаемый результат**	**Результат**
1	Успешный импорт	sample_products.csv	11 строк, 11 импортировано, 0 ошибок; 11 товаров в списке	Выполнено
2	Неверная цена	invalid_price.csv	1 строка, 0 импортировано, ошибка по строке 2: «Поле price должно быть числом больше 0»	Выполнено
3	Дубликат внутри файла	duplicate_sku.csv	2 строки, 1 импортирован, ошибка «Артикул товара уже существует»	Выполнено
4	Повторный импорт	valid_electronics.csv дважды	Второй запуск: 2 строки, 0 импортировано, 2 ошибки дубликата	Выполнено

№	Сценарий	Входные данные	Ожидаемый результат	Результат
5	Пустой файл	empty_file.csv	0 строк, 1 ошибка «Данные для импорта отсутствуют»	Выполнено
6	Неверный is_active	bad_active.csv	Товар не добавлен, ошибка допустимых значений true, false, 1, 0	Выполнено
7	Нет бренда	missing_brand.csv	Товар не добавлен, ошибка по полю brand	Выполнено
8	Гарантия 61 месяц	warranty_out_of_range.csv	Товар не добавлен, ошибка «Гарантия должна быть целым числом от 0 до 60 месяцев»	Выполнено
9	Файл не найден	Несуществующий путь	Ошибка «Файл не найден», приложение не падает	Выполнено
10	Пустой путь	Пустое поле формы	Сообщение «Укажите путь к CSV-файлу»	Выполнено
11	Кириллица	Файл в UTF-8 и в Windows-1251	Названия отображаются без искажений	Выполнено
12	Поиск по бренду	Запрос «Logitech» после импорта sample_products.csv	В таблице 2 товара (артикулы 1002 и 1007)	Выполнено

Доступность операций в зависимости от состояния приложения приведена в таблице 11.

Таблица 11 – доступность операций

Операция	Условие доступности	Метод
Импорт CSV-файла	Всегда	ProductImportService.ImportFromCsv()
Просмотр списка товаров	Всегда	IProductRepository.GetAll()
Поиск, фильтр, сортировка	В памяти есть товары	ProductFilter.Apply()

Операция	**Условие доступности**	**Метод**
Очистка списка	В памяти есть товары	IProductRepository.Clear()
Повторный импорт	Всегда; дубликаты пропускаются	ProductImportService.ImportFromCsv()

ЗАКЛЮЧЕНИЕ

В ходе выполнения практики разработан ASP.NET Core MVC-модуль импорта товаров из CSV-файла для электронного магазина (вариант 1 «Электроника»). Модуль читает файл, разбирает строки с поддержкой кавычек и пустых полей, проверяет все поля по заданным правилам, добавляет корректные товары в хранилище в памяти, формирует результат импорта со списком ошибок и номерами строк и показывает загруженные товары на отдельной странице.

В работе применены слоистая организация кода и разделение ответственности: контроллер остаётся тонким, бизнес-правила сосредоточены в сервисе импорта, хранение скрыто за интерфейсом репозитория, регистрация репозитория как singleton обеспечивает сохранность данных между запросами. Обработка ошибок построена так, что некорректный файл, пустой файл, отсутствующий файл, неверные значения полей и дубликаты артикулов не приводят к падению приложения, а повторный запуск импорта возможен в любой момент.

Разработан комплект из 47 тестовых методов xUnit (54 тестовых случая), покрывающий репозиторий, парсер CSV, сервис импорта, готовые CSV-файлы и функции списка товаров. Подготовлены сценарии ручного тестирования через браузер. Дополнительно реализованы поиск, фильтрация по категории, сортировка, подсчёт итогов, очистка списка и поддержка кодировки Windows-1251.